Amelia Farrell
Term Project – Write-up
DSC650-T301 Big Data
2/17/24

Solar Proton Events (Write-up)

Data Processing through Nifi, HDFS, Spark, and HBase

Introduction

Solar flares are intense bursts of radiation coming from the release of magnetic energy from the sun. Solar flares are classified as the largest explosive events in our solar system today. They can be seen as bright areas of the sun and can last from minutes to hours. While many do not cause us any harm, some can have significant effects on technology and life on Earth (Daglis, I., & colleagues, 2004)[1]. "Therefore, providing accurate and early forecasts of solar flares is crucial for disaster risk management, risk mitigation, and preparedness" (Abduallah, Y., Wang, J.T.L., Wang, H., 2023)[2]. However, predicting these events can be extremely difficult as the buildup happens quickly, giving us little time to make accurate predictions. Scientists have been working on finding better ways to make these predictions for years with no avail. However, with the advent of real-time data processing through tools like Spark, we have the opportunity to develop live data pipelines, making predictions as they happen. We can ingest the data, process it, run it through statistical models, and output insights and notifications automatically. With the availability of large amounts of flare-related data, researchers have already started creating machine learning models built specifically for solar flare forecasting (Abduallah, Y., Wang, J.T.L., Wang, H., 2023)[2]. By combining real-time data processing and machine learning, we can aid researchers in predicting the largest events in our solar system and protecting our infrastructure and lives.

In order to tackle this problem, we have created a data processing flow (big data ecosystem) to analyze solar flare data and predict it's strength (PFU). By using indicators such as day, time, year, and location we can predict where the most impactful (highest PFU) solar flares are likely to occur and when. We accomplished this by using tools such as NiFi, HDFS, Hive, Spark, Spark MLlib, Docker, and HBase. We will dig into the details on how this was done and the outcome in this review.

Data

The data chosen for this architecture comes from the NOAA Space Weather Prediction Center on past solar events. The data ranges from May of 1976 to September of 2024. It consists of details on over 300 recorded events of varying magnitudes. The details include, date, start time, end time, duration of event, max pfu (magnitude), location, class, sun impact, and more. While the data set has a lot of features, many transformations had to take place before it was ready for modeling. These transformations will be discussed later in this review.

Note: The data was copied and saved on GitHub for easy access and to replicate a live csv data feed in our architecture. Links to data source provided on the last page of this review.

Method (Pipeline Overview)

Our data pipeline for ingesting, analyzing, and predicting solar flare data involved the following:

1. **Environment**

   Our dockerized environment consisted of Hadoop, Hive, Spark, and HBase. It also consisted of multiple docker containers (master, worker1, worker2, namenode, HBase, spark) made accessible within the same network.

2. **NiFi Data Ingestion**

   As noted earlier, data ingestion was done using a raw csv file hosted on GitHub. In order to mimic a real-time data ingestion flow, we used NiFi to pull the data from the webpage and write it to HDFS.

   To start NiFi, we went the directory within our dockerized environment (cd dsc650-infra/bellevue-bigdata/NiFi) and started NiFi (/bin/bash nifi-*/bin/nifi.sh start). We then created a processor group containing InvokeHTTP and PutHDFS processors. The InvokeHTTP processor pulled the data from the webpage and the PutHDFS processor stored it in HDFS.

3. **HDFS Processing**

   Once the data was written into HDFS, we renamed the file (from a random name such as fffc1dd9-58e8-…) using hdfs dfs -mv to solar_events.csv. This made it easier to identify in our schema.

4. **Hive**

   Next, a Hive session was opened in order to load the data into a Hive table for further processing. The table was named “solar_events” for consistency. We then reviewed our Hive table by using commands such as DESCRIBE, SELECT, and COUNT. To these commands help us view our table schema and count the number of missing values.

5. **Spark MLib** (loading machine learning libraries)

   After exiting Hive, we loaded each of the docker containers and installed the Python package NumPy to each to help us create a streamlined machine learning model in PySpark.

6. **PySpark – Data Processing & Cleaning**

   PySpark was then used to load the solar_events table from Hive and perform data cleaning on the missing values we identified in our Hive session. We also performed further data checks and transformations such as replacing text values in our location features (latitude and longitude) with numeric values, dropping duplicate columns, covering predicting features to floats, and creating a correlation matrix to identify multicollinearity. Necessary for preparing our data for modeling. This was all done within a Spark Session. The Spark Session provides the necessary configuration (e.g., Hive metastore access, HDFS integration) to execute SQL queries and manage DataFrames efficiently.

7. **PySpark – Model Building (Decision Tree Regression Model)**

Once the data is thoroughly cleaned and processed, it is ready for model building. We chose a Decision Tree Regression model to predict the Max PFU (magnitude) in the solar_events dataset. We chose this model due to how it handles non-linear relationships, outliers, and missing data. It also provides us with interpretable feature importance (e.g., location, month, time of day), which is critical for scientific analysis of solar proton events. Its simplicity and scalability makes it ideal for training on new data fed by real-time processing. While Decision Tree Regression risks overfitting with small datasets, we mitigated this by fixing the max depth to 10. In summary, the Decision Tree's efficiency, interpretability, and ability to model the complexity of solar flare data, made it the best choice for our goal.

We were able to build the model in PySpark MLlib by leveraging it's existing library, 'DecisionTreeRegressor'. This allowed us to build and train the model directly in our data ecosystem. Making it perfect for reuse when new data is fed into NiFi.

To train the model, we fist identified the prediction variables and the target (maxflux, aka Max PFU). Then split the data into training and test sets before training the model on the training set. Once trained, we evaluated it performance by looking at R2, MAE, RMSE, and Feature Importance. This was all possible to the availability of the numpy package within our session.

How did the model preform?

- **RMSE** (Root Mean Squared Error): 50.23
- **R2** (R-squared): 0.75
- **MAE** (Mean Absolute Error): 35.67

The RMSE of 50.23 indicates the average prediction error for Max PFU (magnitude of solar flare) is about 50.23 units, which is moderate given the range of values (12 to 850 in our data set).

An R2 of 0.75 means that our model can explain 75% of the variance in Max PFU, suggesting a reasonably good fit but room for improvement.

Our MAE of 35.67 indicates the average absolute error in predictions is about 35.67 units, which is not terrible but could certainly be improved.

So which features were most important for predicting Max PFU? "latdegrees" or Degree of latitude. Meaning that the distance north or south of the earth's equator is the strongest predictor of a solar event's intensity/Max PFU.

- **latdegrees**: 0.466 (most important)
- **year**: 0.334 (second most important)
- **logdegrees**: 0.091
- **region**: 0.045
- **maxday**: 0.033
- **month**: 0.025

To summarize, our model performed well considering that no feature smoothing or cross validation was done. While an error of 50.23 is high, an R2 of 0.75 is a good starting place. With the addition of smoothing, GridSearch, and 5-fold cross validation, I can see the path to a model with a much lower error rate and increased accuracy. Our model also uncovered the strong relationship between the degree of latitude and solar flare intensity. Giving us something to investigate further.

8. **HBase (creating performance metrics table)**

   After model creation and testing, we want to store the performance metrics (RMSE, R2, MAE) and feature importance in HBase for future access. This will give us a way to compare the next models and see the performance improve. We are able to store the metrics in HBase using happybase in PySpark. Before doing so we installed 'happybase' in each of our docker containers. Once installed, we could enter the master bash, open the HBase shell, and create the 'performance_metrics' table with a 'stats' column family to store our data.

9. **PySpark – (writing the metrics to HBase table)**

   Since the data cannot be stored in the new table via HBase. We needed to, re-entered PySpark, imported the 'happybase' package, define the performance metrics (by assigning them to variables), and write them to the 'performance_metrics' table in HBase. We could then retrieve the performance metrics data in the new table from HBase using the scan command in the HBase shell.

What Worked

We successfully initialized the Docker environment using docker-compose up -d, ensuring Hadoop, Hive, Spark, and HBase containers communicated via a shared network. Allowing for seamless interaction between components. NiFi worked well for data ingestion, successfully pulling data from GitHub webpage and writing it to HDFS (with processors InvokeHTTP and PutHDFS). HDFS and Hive worked to create/loading the solar_events table in Hive, enabling initial data exploration. PySpark was effective at cleaning and transforming the data as needed as well as at building and testing our chosen model. Lastly, HBase was able to create a table to store our models performance metrics.

Challenges / Areas for Improvement

Aside from the improvements needed to our machine learning model, we ran into some issues such as 'JAVA_HOME not set' warning when starting NiFi. We were not able to fully resolve the problem which could potentially affecting stability of our data flow. Starting Hive also gave us some warnings indicating potential classpath conflicts, but queries still executed successfully. PySpark was also limited due to our version. However, a workaround was possible without needing to update. We also ran into some strange R values when running our correlation matrix. With 'maxday' vs. 'logdegrees' having an R of 9.9 (far outside of the -1 to 1 expected range). While 'maxday' was not needed for our model, it was removed, also removing the need for a solution.

Conclusion

Through our data processing flow consisting of Nifi, HDFS, Hive, Spark, and HBase. We successfully processed solar proton event data and completed a machine learning model for predicting Max PFU. While there are still improvements that can be made, I believe this architecture shows just how powerful these tools can be for data steaming, processing, and storing. These attributes make it the ideal fit for scientists looking to predict solar flares in real-time. I look forward to making future improvements and starting in ingest of live solar data into the production ecosystem.

A full walkthrough with code and screen shots can be found in the accompanying PDF

Data Source

- Original: National Centers for Environmental Information. (n.d.). *Solar Proton Events Affecting the Earth Environment*. NOAA Space Weather Prediction Center. https://www.ngdc.noaa.gov/stp/space-weather/interplanetary-data/solar-proton-events/SEP%20page%20code.html

- Github: https://raw.githubusercontent.com/Amelia-ZJ/DSC650/refs/heads/main/solar_proton_events_cleaned.csv

References

1. Daglis, I., Baker, D., Kappenman, J., Panasyuk, M. & Daly, E. Effects of space weather on technology infrastructure. *Space Weather* **2**, S02004 (2004).
2. Abduallah, Y., Wang, J.T.L., Wang, H. *et al.* Operational prediction of solar flares using a transformer-based framework. *Sci Rep* **13**, 13665 (2023). https://doi.org/10.1038/s41598-023-40884-1
3. Daglis, I., Baker, D., Kappenman, J., Panasyuk, M. & Daly, E. Effects of space weather on technology infrastructure. *Space Weather* **2**, S02004 (2004).